

Visualization Systems and Toolkits

Arjun Srinivasan

IDC School of Design | Sep 2026

Agenda

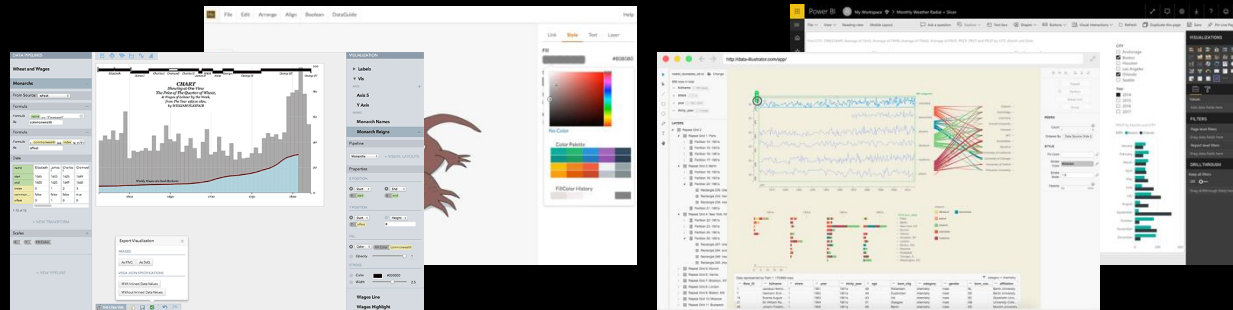
- Walkthrough different categories of visualization systems and toolkits
- Demonstrate how an understanding of tools and interaction/intent categories (from the previous talk) can be used in practice.

Takeaways

- Dimensions to consider when deciding on which visualization system/toolkit to use for a project or task.
- An appreciation for (/interest in) visualization research



Visual analysis and reporting tools



Bespoke chart authoring tools

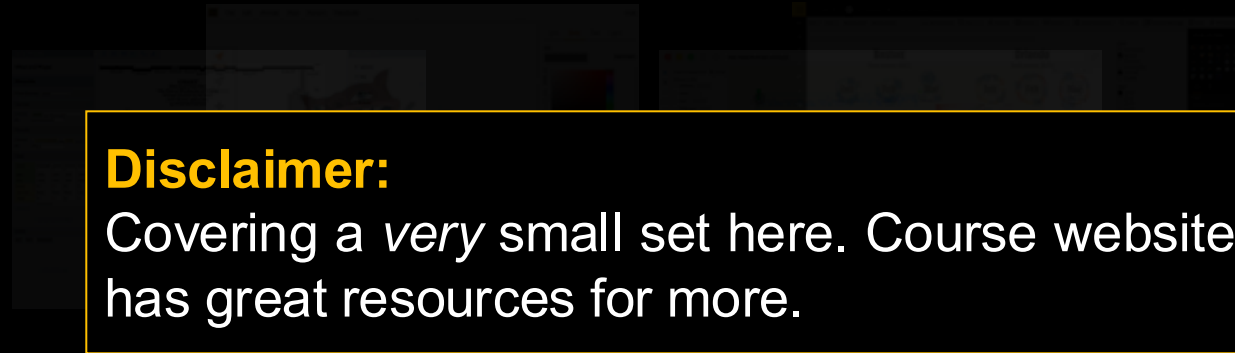


Vega-Lite

Programming toolkits and grammars



Visual analysis and reporting tools



Disclaimer:

Covering a very small set here. Course website has great resources for more.

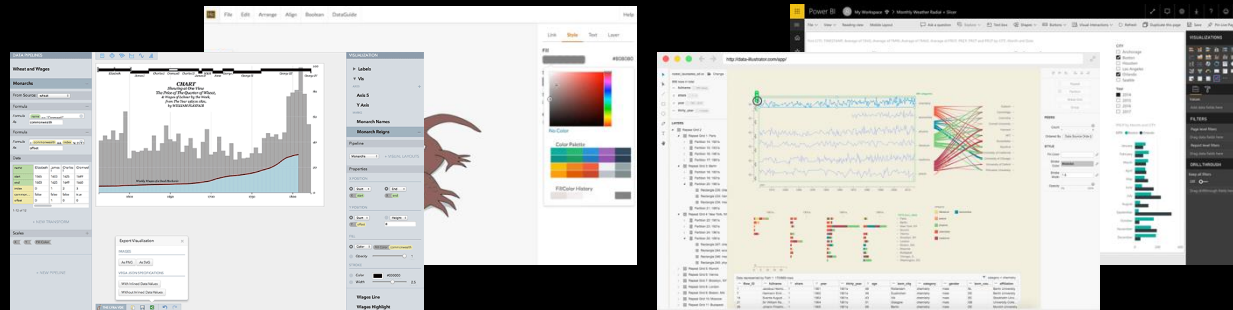
Bespoke chart authoring tools



Programming toolkits and grammars



Visual analysis and reporting tools



Bespoke chart authoring tools



Vega-Lite

Programming toolkits and grammars



Visual analysis and reporting tools

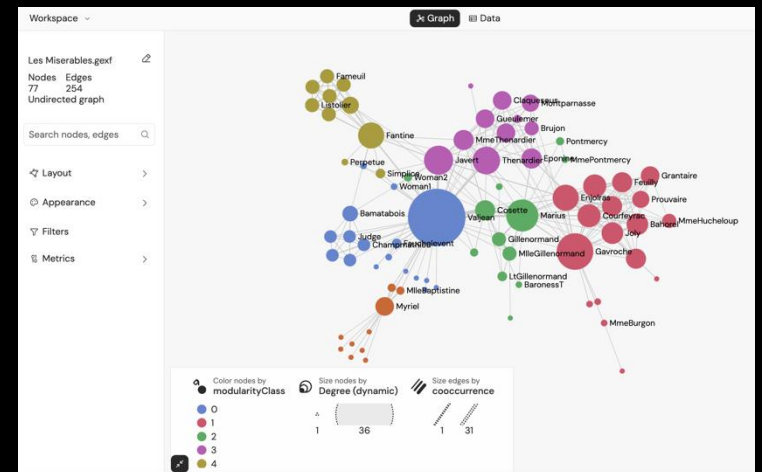
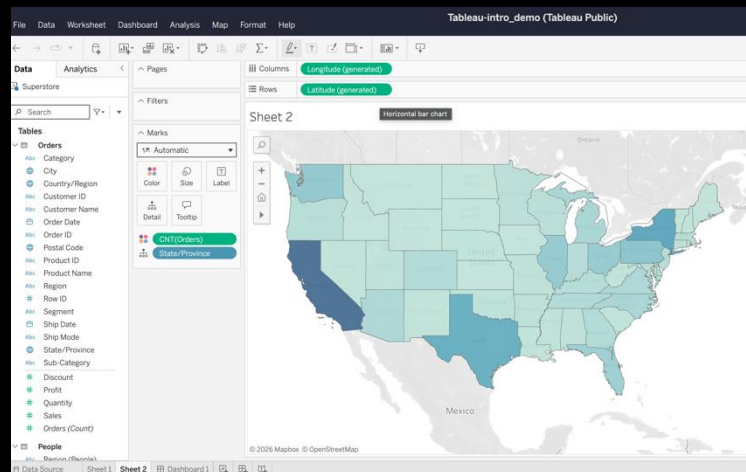
Why:

- SQL and analysis needed expertise
- Data stack-specific chart authoring

Target users:

- Data/BI analysts, consultants

Programming toolkits and grammars





Visual analysis and reporting tools



Pros:

- Interactive experience
- Rich authoring defaults
- Layered customization support

Challenges:

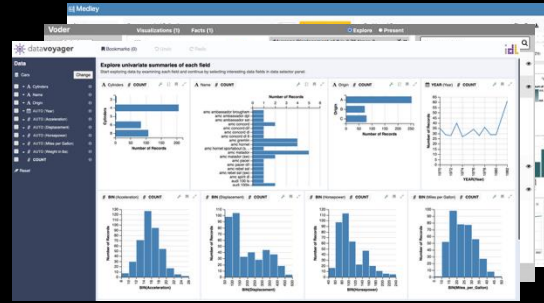
- Expertise
- Incremental/sequential flow

Bespoke chart authoring tools



Vega-Lite

Programming toolkits and grammars



Visual analysis and reporting tools

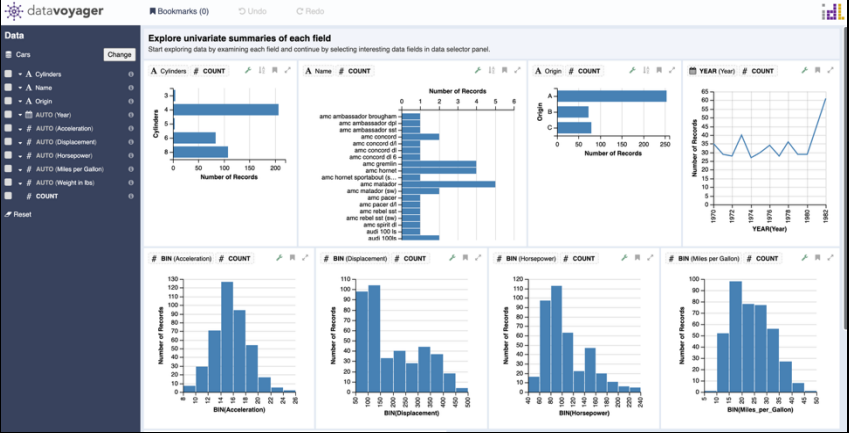


Bespoke chart authoring tools



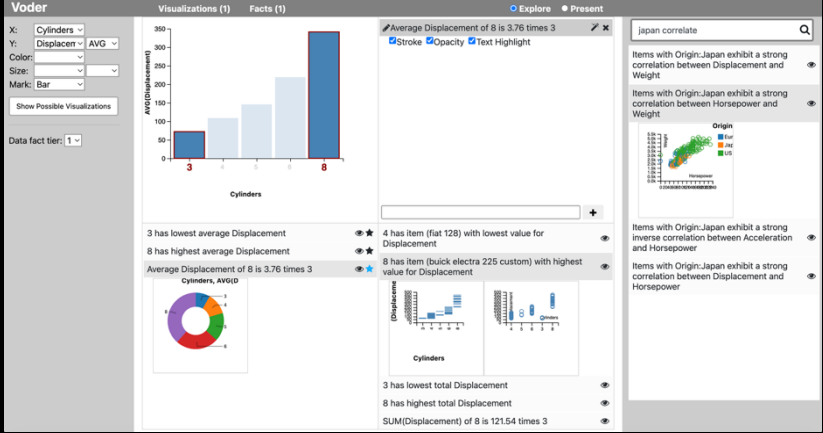
Vega-Lite

Programming toolkits and grammars



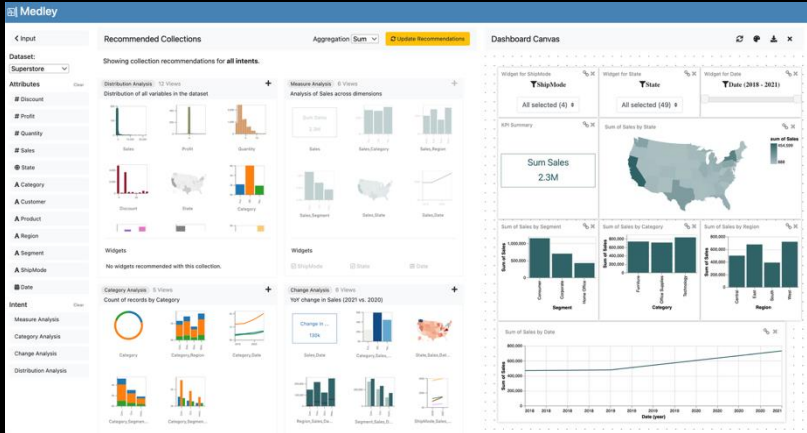
Data Voyager

Wongsuphasawat et al., 2015, 2017



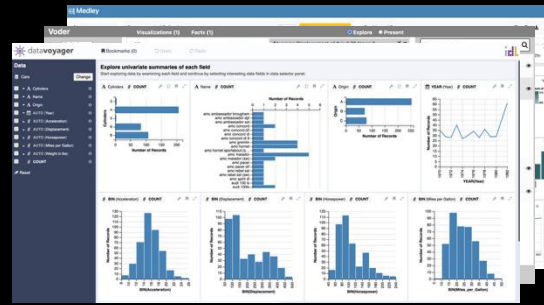
Voder

Srinivasan et al., 2018

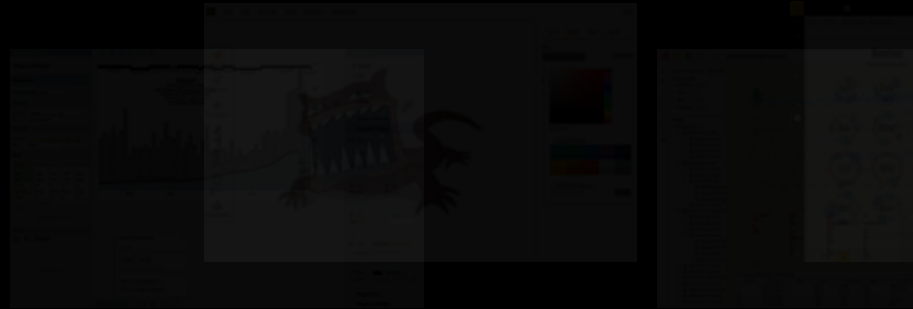


Medley

Pandey et al., 2022



Visual analysis and reporting tools



Bespoke chart authoring tools



Vega-Lite

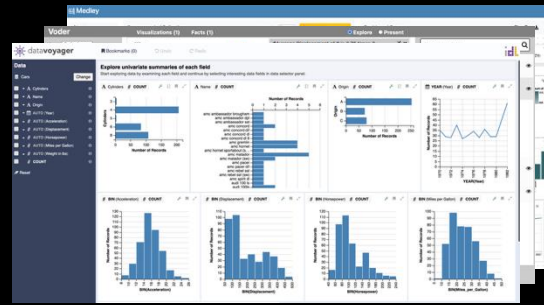
Programming toolkits and grammars

Pros:

- Rapid, iterative workflow with fewer decisions required on the users' part.

Challenges:

- Relevance of suggestions and steering system output



Visual analysis and reporting tools



Bespoke chart authoring tools



Vega-Lite

Programming toolkits and grammars

Tableau

Power BI



Visual analysis and reporting tools



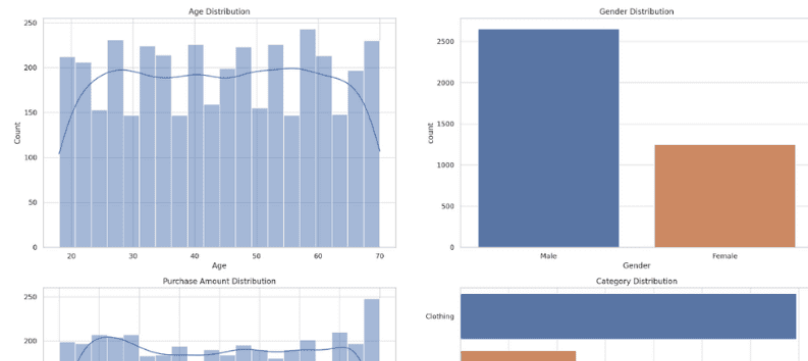
Bespoke chart authoring tools



Vega-Lite

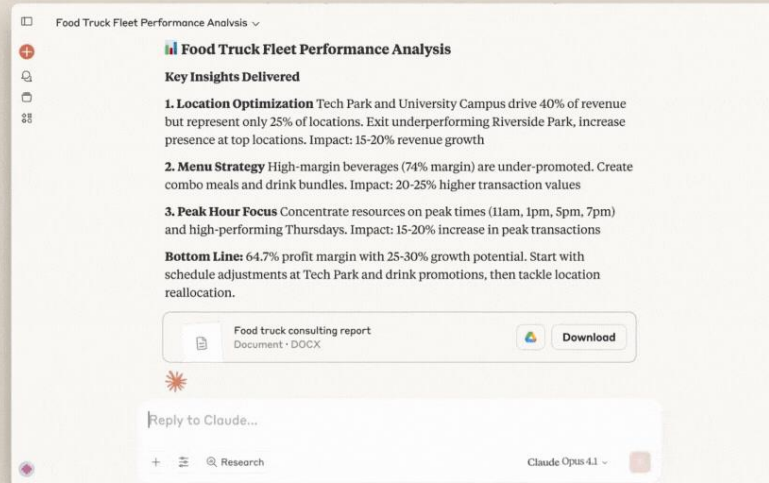
Programming toolkits and grammars

ChatGPT



shopping_trends.csv
Spreadsheet

Perform exploratory data analysis on customer shopping trends dataset and display only plots.



Pros:

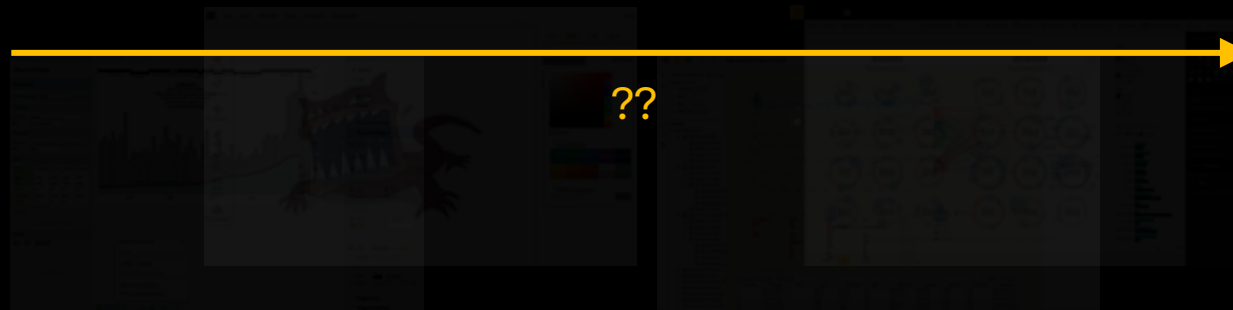
- Minimal effort and expertise

Challenges:

- Validation, relevance, iteration



Visual analysis and reporting tools



Bespoke chart authoring tools

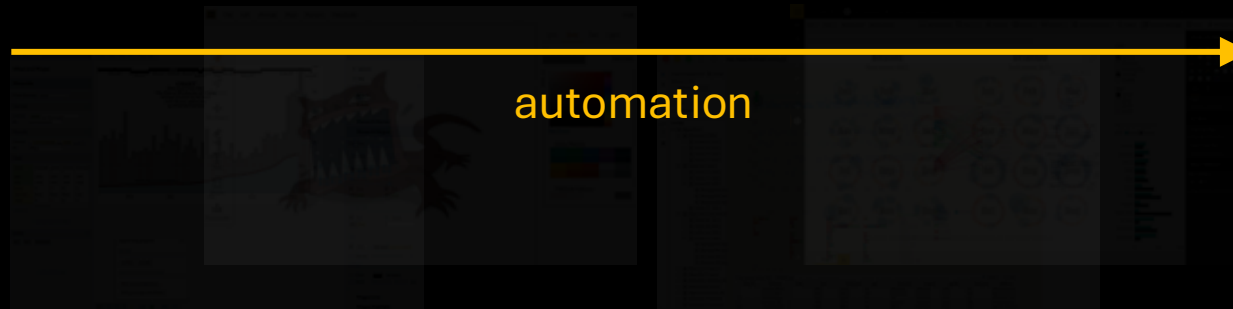


Vega-Lite

Programming toolkits and grammars



Visual analysis and reporting tools



Bespoke chart authoring tools



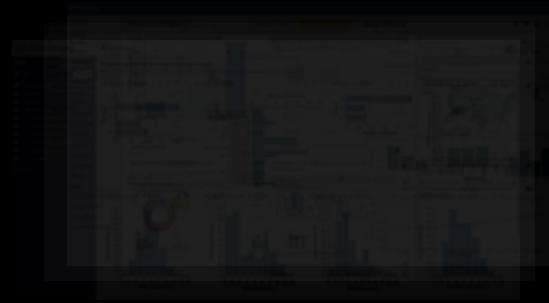
Vega-Lite

Programming toolkits and grammars

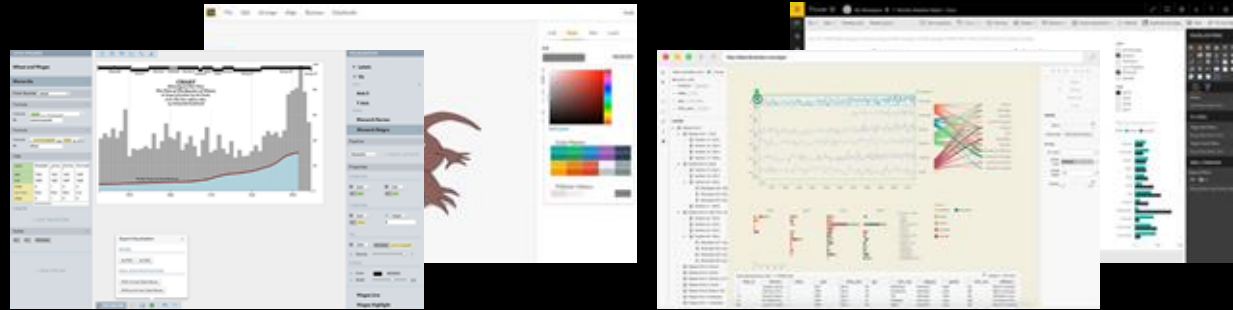
Tableau



Power BI



Visual analysis and reporting tools

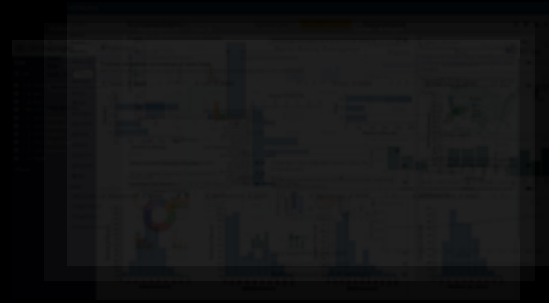


Bespoke chart authoring tools

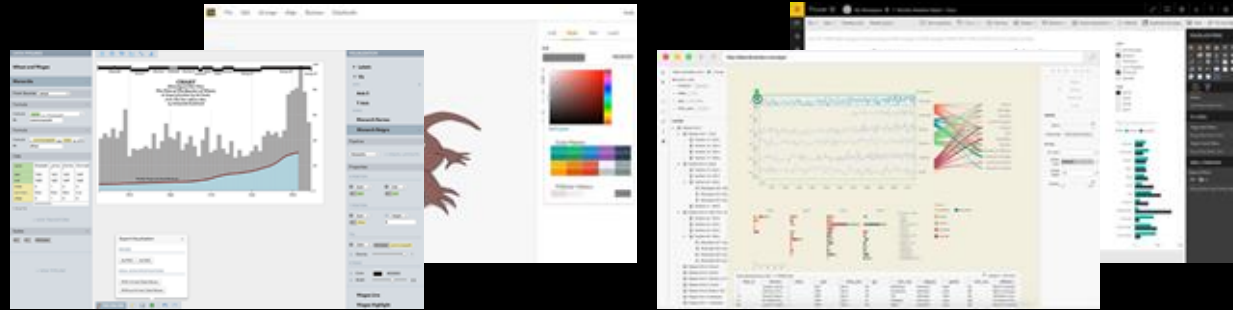


Vega-Lite

Programming toolkits and grammars



Visual analysis and reporting tools



Bespoke chart authoring tools



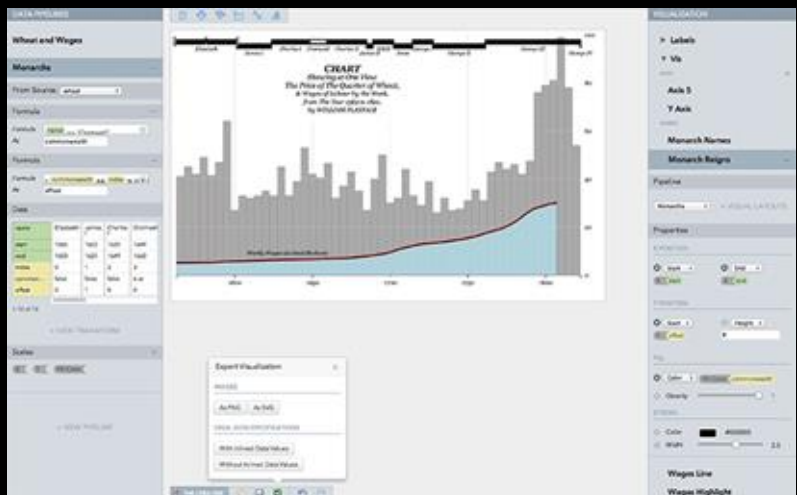
Programming toolkits and grammars

Why:

- Creating custom graphics (think Infographics) is difficult
- Designers already have a tool stack

Target users:

- Designers (esp. those in journalism teams)



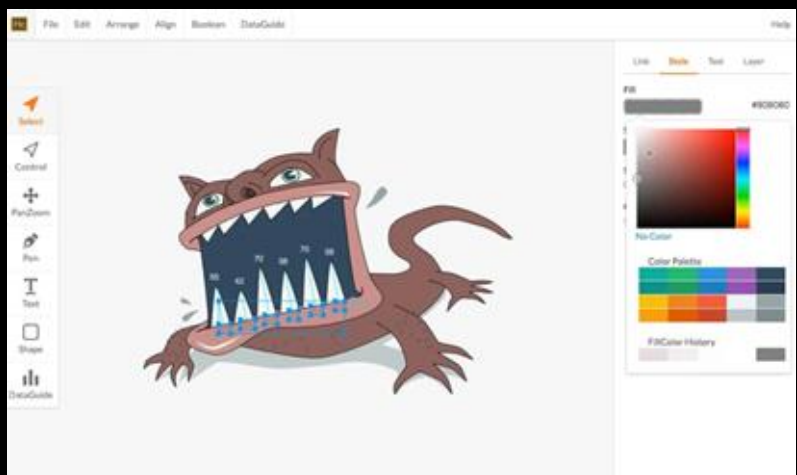
Lyra

Satyanarayan and Heer, 2014



Data Illustrator

Liu, Thompson et al., 2018



Data-Driven Guides

Kim et al., 2016



Charticulator

Ren et al., 2018



Lyra

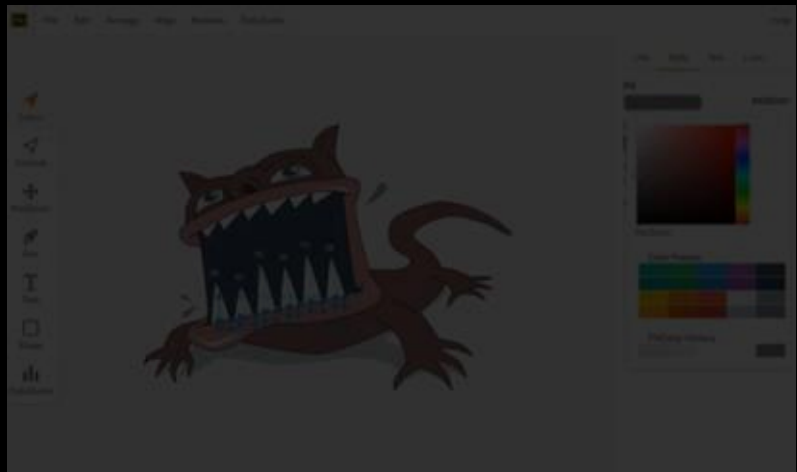
Satyanarayan and Heer, 2014



Data Illustrator

Liu, Thompson et al., 2018

([example](#)) [2.30-4.30]



Data-Driven Guides

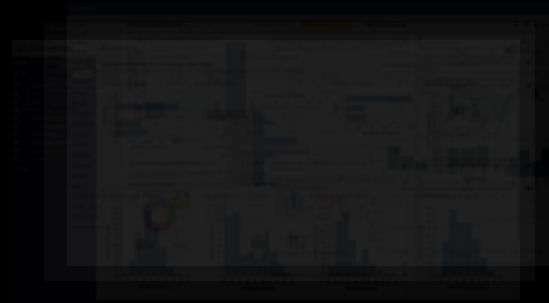
Kim et al., 2016

([example](#)) [0.38-2.30]

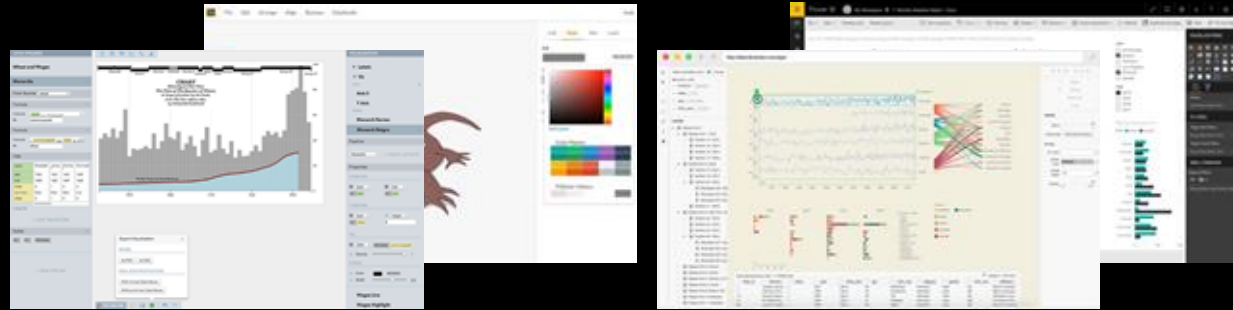


Charticulator

Ren et al., 2018



Visual analysis and reporting tools



Bespoke chart authoring tools



Programming toolkits and grammars

Pros:

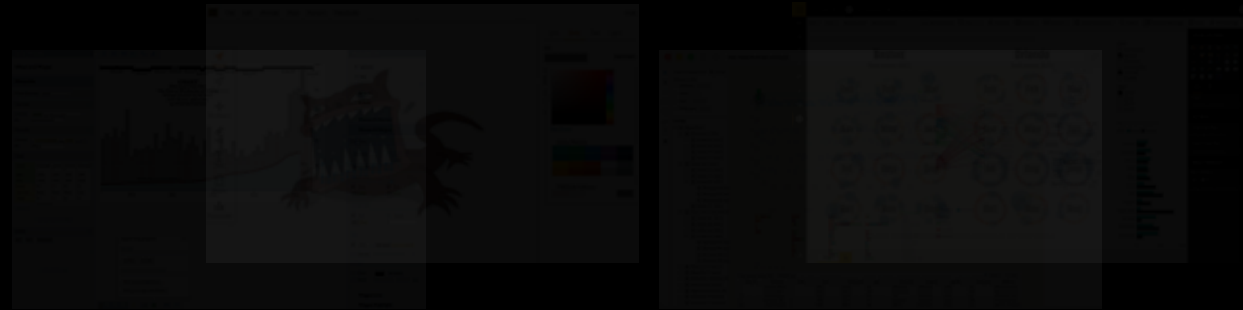
- Customization
- Data integration into design tools

Challenges:

- Learning curve
- Bounded by the tool's features/workflow



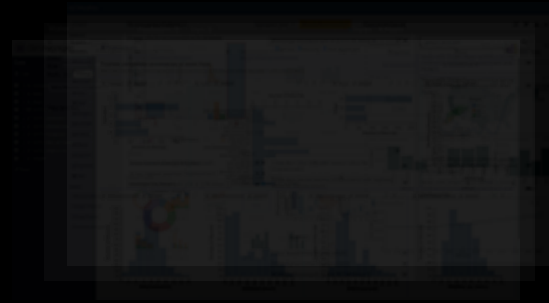
Visual analysis and reporting tools



Bespoke chart authoring tools



Programming toolkits and grammars



Why:

- Customization is bounded by tool
- Application building (need these to build the other tools we've seen so far)

Target users:

- Programmers (...a fading term with LLMs)



Vega-Lite

Programming toolkits and grammars



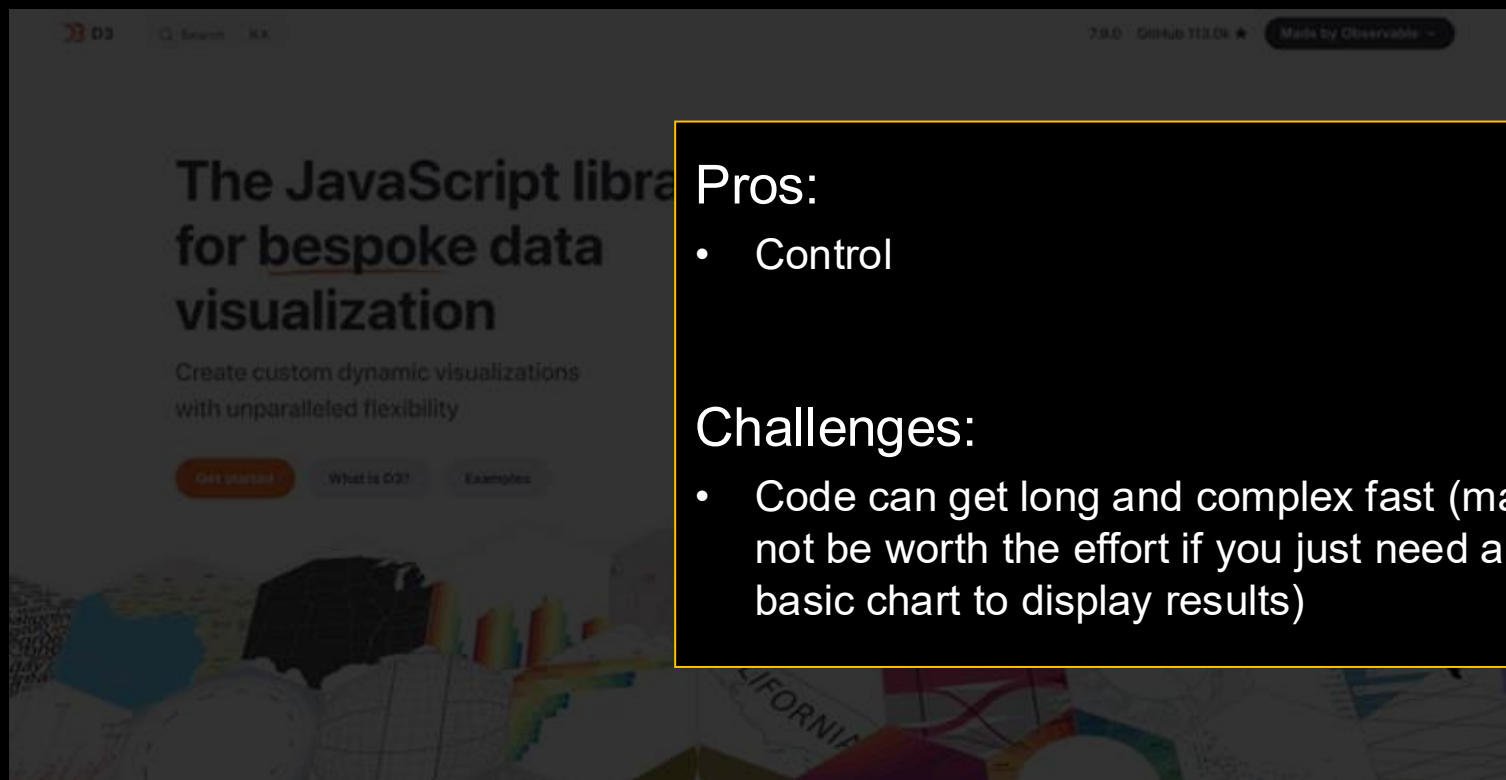
Data-Driven Documents



The screenshot shows the D3.js website homepage. At the top, there is a navigation bar with the D3.js logo, a search bar, and links to GitHub (113.0k stars) and the Observable project. The main heading reads "The JavaScript library for bespoke data visualization". Below this, a subheading states "Create custom dynamic visualizations with unparalleled flexibility". There are three buttons: "Get started" (highlighted in orange), "What is D3?", and "Examples". The bottom of the page features a collage of various data visualizations, including maps, bar charts, pie charts, and network diagrams.

[Basic example](#)

[Basic example \(+interaction\)](#)



Pros:

- Control

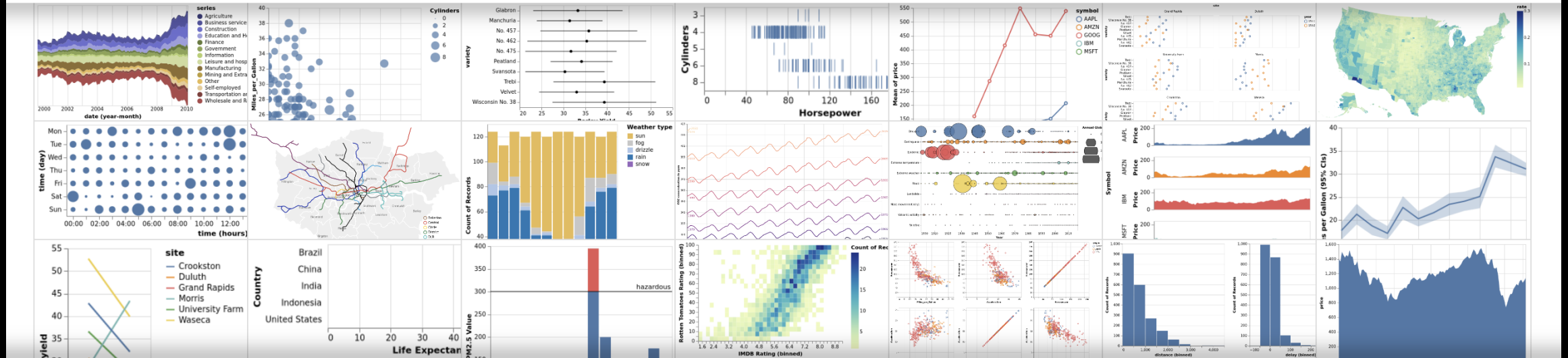
Challenges:

- Code can get long and complex fast (may not be worth the effort if you just need a basic chart to display results)

[Basic example](#)

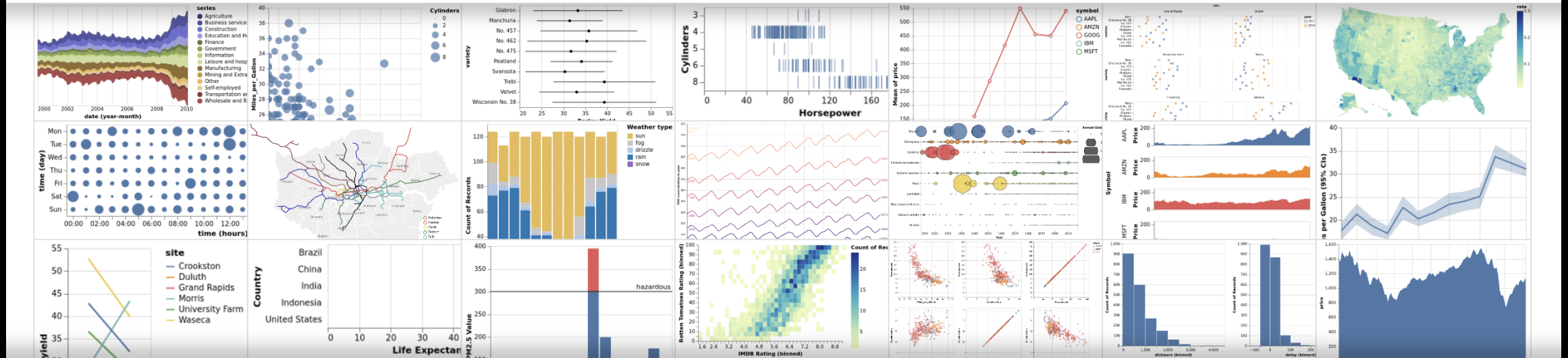
[Basic example \(+interaction\)](#)

Vega-Lite – A Grammar of Interactive Graphics



Vega-Lite is a high-level grammar of interactive graphics. It provides a concise, declarative JSON syntax to create an expressive range of visualizations for data analysis and presentation.

Vega-Lite – A Grammar of Interactive Graphics



Vega-Lite is a high-level grammar of interactive graphics. It provides a concise, declarative JSON syntax to create an expressive range of visualizations for data analysis and presentation.

- Less programming, more “specification” (*declarative programming*)
- Particularly easy to use when building systems

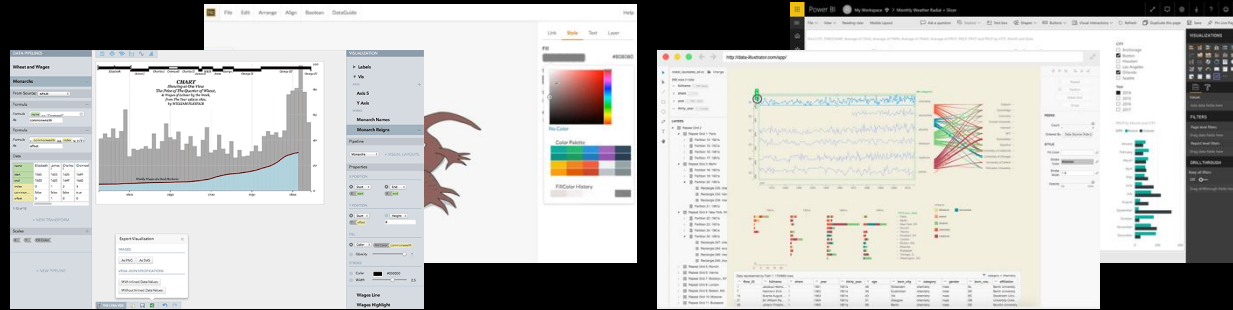


Basic bar example

Basic bar example (+interaction)



Visual analysis and reporting tools



Bespoke chart authoring tools

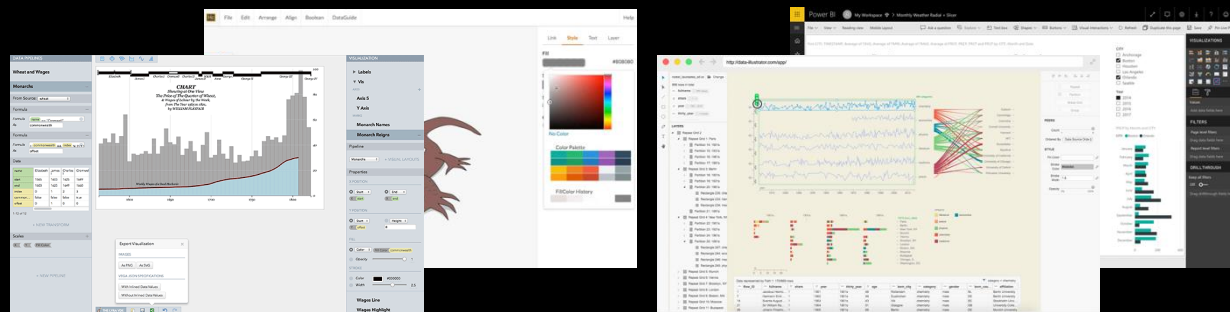


Vega-Lite

Programming toolkits and grammars



Visual analysis and reporting tools



Bespoke chart authoring tools



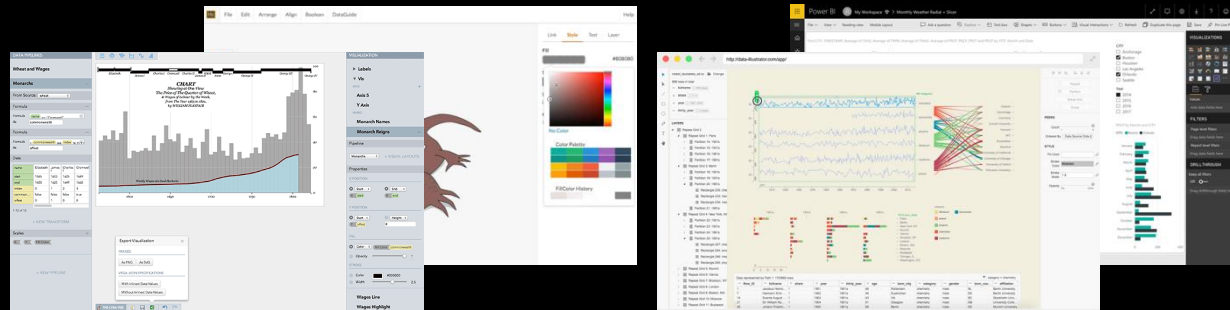
Vega-Lite

Programming toolkits and grammars





Visual analysis and reporting tools



Bespoke chart authoring tools



Vega-Lite

Programming toolkits and grammars

Customization

Scenarios: What would you choose?



- Have a dataset and need a basic dashboard to share findings
- Designing/developing a system that generates charts as part of the UI
- Creating an Infographic for a news outlet



Vega-Lite

Lyra, Data Illustrator, etc.

Scenarios: What would you choose?

- Have a dataset and need a basic dashboard to share findings



- Designing/developing a system that generates charts as part of the UI



- Creating an Infographic for a news outlet



How does any of this matter if I'm just going to use a LLM to build my app?

Explore

Upload a table. Charts are chosen from the shape of the data.

[Load sample data](#)[Upload CSV](#)**DATASET****movies.csv**

ROWS

503 / 503

COLUMNS

11

CHARTS

4

FILTERS

0**FILTERS**[Clear](#)

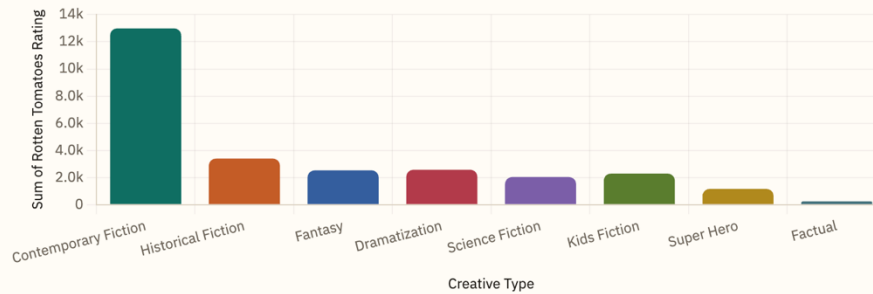
Click any chart mark to cross-filter.

COLUMNSidx **NUMERIC**Title **TEXT**Worldwide Gross **NUMERIC**Production Budget **NUMERIC**Release Year **NUMERIC**Content Rating **CATEGORICAL**Running Time **NUMERIC**Major Genre **CATEGORICAL**Creative Type **CATEGORICAL**Rotten Tomatoes Rating **NUMERIC**IMDB Rating **NUMERIC**

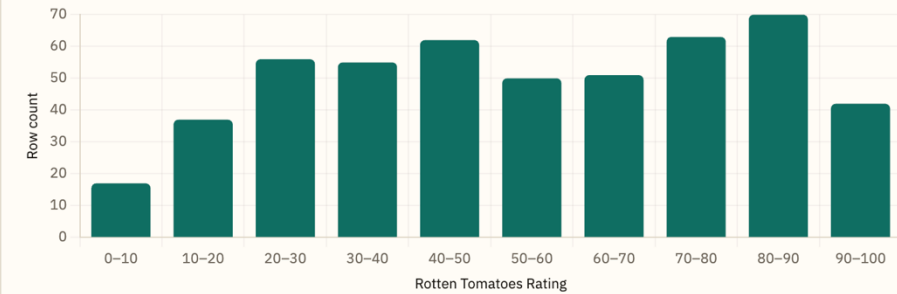
Click a bar, slice, point, or histogram bin to cross-filter the whole dashboard.

Rotten Tomatoes Rating by Creative Type

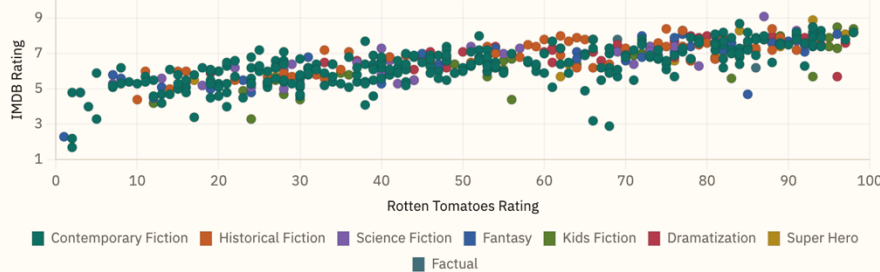
Sum of Rotten Tomatoes Rating for each Creative Type. Click a bar to filter the dashboard.

**Distribution of Rotten Tomatoes Rating**

Binned Rotten Tomatoes Rating to show shape and outliers. Click a bin to keep that range.

**IMDB Rating vs Rotten Tomatoes Rating**

Each row is a point, colored by Creative Type. Click a point to keep that Creative Type.

**Share of Content Rating**

Part-to-whole view of Content Rating. Click a slice to isolate it.



Write a basic web app (HTML+JS+CSS) for interactive visual data exploration. App requirements:

1. Allows uploading a CSV file
2. Automatically generates 3-5 charts using the uploaded data and presents them as a dashboard
3. Supports interacting with the charts to explore data

The scatterplot interactions need to be updated:

1. Selecting individual points should be allowed (not only the underlying category). The legend selection should stay.
2. Zooming should be supported so users can find and select points of interest

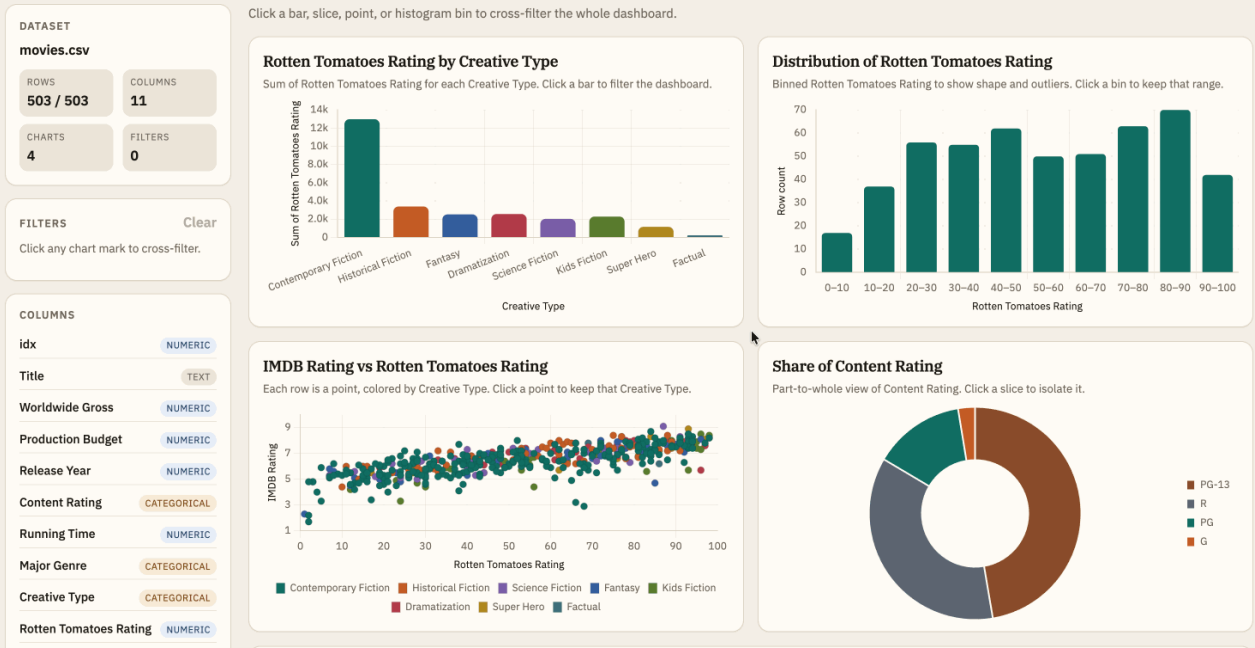
A general change to the crossfiltering: all charts should support multi-select selections via shift/cmd+click

****only for a quick demo; by no means are these exemplary design/development prompts**

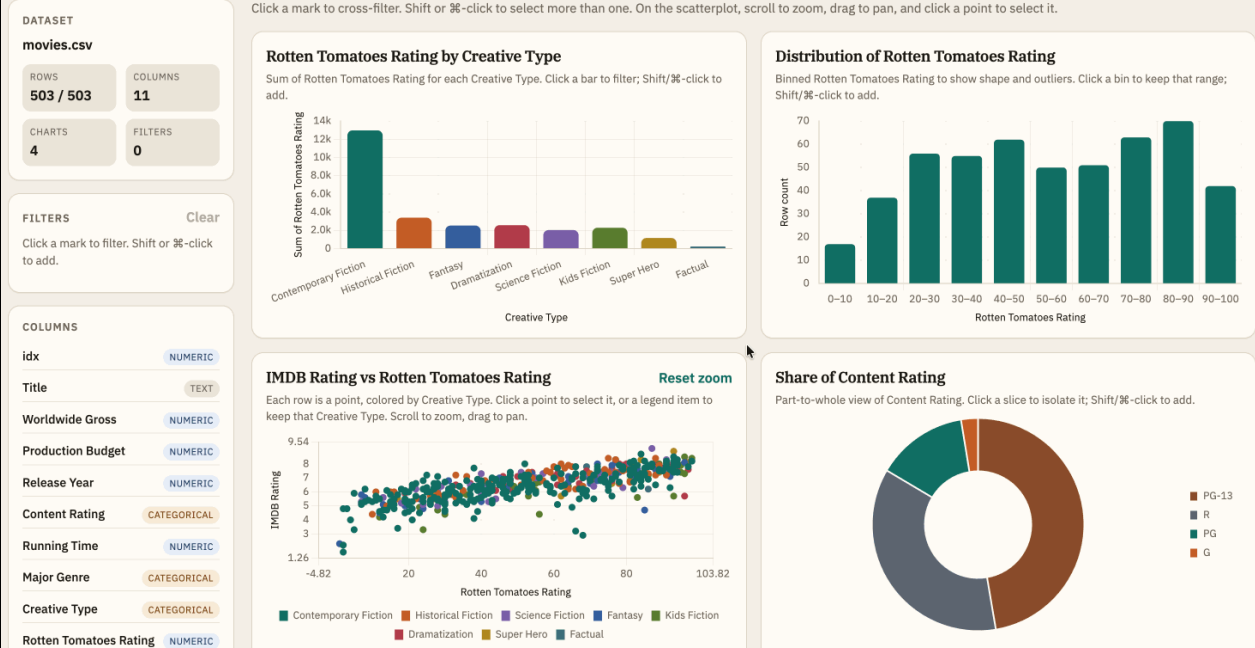
- when filtered, target chart scales should update to only show active values.

- during crossfiltering, source charts should not reset during selection (e.g., when I am zoomed into the scatterplot and select a point, the zoom focus resets)

both legend and individual mark selection should be allowed



One-shot



Updated interactions to support richer analytic intents

If you had to build this version of the app, would you use a different library?

Why did you use chart.js?

No. For this version I would not start with Chart.js. I used it because it was already in the app: Papa Parse + Chart.js, canvas charts, click-to-filter. The work was to extend that stack, not replace it.

Chart.js is a **chart widget** library. It is good at drawing a bar, line, doughnut, or scatter and attaching a click handler. This version is a **coordinated-view explorer**: multi-select, legend vs mark selection, scatter zoom that must survive selection, source charts that stay put, target charts that refit. Those are system behaviors. Chart.js does not have them, so they were built around the library — in-place updates so zoom is not destroyed, extra plugins for pan/zoom, hit-testing so legend clicks are not stolen, row IDs because points have no identity, OR vs AND for legend and points.

If I were building this version from scratch, I would use **Vega-Lite** (via Vega-Embed). Point and interval selections, toggle with Shift, scales that follow filtered data, and linked views are part of the model, not plugins. For an infovis systems course that is the better fit: the spec is the system design.

Plotly would be the next choice if the priority were a polished scatter (zoom, pan, lasso) with less interaction theory. **D3** if the assignment is to implement scales, selection, and linking yourself.

I would still keep Chart.js for a simpler “recommend a few charts from a CSV” demo. Once the product is a crossfiltering dashboard with a zoomable scatter, it is the wrong layer.

Recap + Takeaways

- Intent-focused interaction categories can be useful for testing and prompting during AI-assisted system development
- Choice of tool can significantly impact the user experience a system can afford. LLMs are great at writing code but need explicit guidance on user and system goals.

Recap + Takeaways

- Intent-focused interaction categories can be useful for testing and prompting during AI-assisted system development

- Choice of tool can significantly impact the user experience. the grouped bar chart failed and the precise visualization vocabulary to instruct the AI on the appropriate fix. Automated tools do not eliminate traditional skills; rather, they transform them. Chart authoring skills must evolve to encompass code verification, targeted prompting, structural critique, and the ability to steer AI agents away from conceptually flawed defaults and toward appropriate solutions.

The CAPED Framework: Characterizing User Skills in Chart Authoring through the Lens of Tasks and Tools
Lee et al., 2026 (published yesterday/today)

Acknowledgements

- John Stasko, Alex Endert. Georgia Tech
- Arvind Satyanarayan. Massachusetts Institute of Technology
- Kanit Ham Wongsuphasawat. Databricks

Thank you

- Email: arjun.srinivasan.10@gmail.com
- AMA chats (“office hours”): bit.ly/arjunsrinivasan-office-hours

Website: <https://arjun010.github.io/>